



ACTAI • WEEK 04 • LECTURE 03

Logins & the Full Build

Authentication, per-user data — and a full end-to-end app that ties the whole week together.

Concepts + a complete capstone build

THE OPEN DOOR

Our app has no security yet



Firestore in 'test mode' lets ANYONE read and write everything. There are no accounts and no privacy.



Who are you?

the app can't tell users apart



Whose data?

everyone shares one big pile



Today's fix

logins + per-user + rules

THE CONCEPT

What authentication is

Authentication is how your app answers one question: 'who are you?' — so it can keep each person's data separate and safe.



Sign up

create an account (email + password)



Log in

prove it's you next time



Session

the app remembers you're in



Log out

end the session safely

TWO WORDS THAT SOUND ALIKE

Authentication vs Authorization



Authentication

WHO you are. Proving your identity by logging in.

"Show me your ID at the door."



Authorization

WHAT you're allowed to do once you're in.

"You can enter these rooms, not those."

In our app: logging in = authentication; seeing only YOUR inventory = authorization.

WAYS TO LOG IN

Sign-in methods



Email & password

the classic; works everywhere



Google / social

one tap, no new password



Magic link

a login link emailed to you



Phone / SMS code

great for mobile (costs at scale)



Anonymous / guest

try first, sign up later



2-factor (MFA)

extra security for sensitive apps

For class we'll use email/password and Google — both free and simple.

OUR TOOL

Firebase Authentication



It pairs perfectly with the Firestore database you already set up — same project, same console.



Free

email & social free for ~50k users



Many methods

email, Google, phone, more



Secure

Google handles passwords safely



Knows the user

gives you a unique user ID

STEP BY STEP

Turn on Authentication

1. In the Firebase console → **Authentication**
2. Click **Get started**
3. Open the **Sign-in method** tab
4. Enable **Email/Password**
5. (Optional) enable **Google** too
6. Save — your app can now have logins



That's the whole setup

Flip on the methods you want — Firebase handles the hard parts (storing passwords, sessions, resets).

IN YOUR APP

Add sign up & log in

Back in your vibe-coding tool, just describe the login flow. The tool wires up Firebase Auth for you.

TELL YOUR APP

Add user accounts with Firebase Authentication.

Let users sign up and log in with email & password (and Google).

Show a login screen first. After login, show the app with a

"Log out" button in the corner.



After login, the app knows the user's unique ID (auth.uid). That ID is the key to keeping each person's data separate — next slide.

THE KEY MOVE

Per-user data: my data $\neq$ your data

Stamp every record with the owner's user ID. Then the app only loads records that belong to the logged-in user.

EACH ITEM CARRIES ITS OWNER

```
{ item: "Tomatoes", quantity: 12, reorderLevel: 5,  
  userId: "aX9k..." ← who this belongs to }
```

THE APP ONLY ASKS FOR YOUR ITEMS

```
load items where  userId == currentUser.uid  
// so you never see anyone else's inventory
```

LOCK IT DOWN

Security rules replace 'test mode'

Filtering in the app is polite — rules make it enforced. These live in Firestore and can't be bypassed.

FIRESTORE RULES — only your own data

```
match /inventory/{doc} {  
  allow read, write: if request.auth != null  
    && request.auth.uid == resource.data.userId;  
}
```

In plain English: you must be signed in, AND the item's userId must match yours — or access is denied.

THE CAPSTONE

One app, the whole week

Build the complete, real, multi-user app — it uses everything: database + CRUD + auth + per-user + real-time.

FULL BUILD PROMPT — paste into your tool

Build a complete inventory app with Firebase [config: PASTE].

- Sign up & log in with email/password and Google (Firebase Auth); show a login screen.
- Each user sees only THEIR OWN items. Store each item's userId.
- Firestore "inventory" collection: item, category, quantity, unit, reorderLevel, userId.
- Full CRUD: add, view, edit, delete. Highlight low stock (quantity <= reorderLevel) in red.
- Update in real-time. A "Log out" button. Clean, modern design with a header.

THE FULL RECIPE

What the capstone put together

1

Design the data

decide the fields (columns)

2

Pick a database

Sheet → Firestore

3

Connect it

give the app the config

4

CRUD

add, read, edit, delete

5

Add logins

Firebase Authentication

6

Secure it

per-user data + rules

This six-step recipe is how essentially every data-driven app gets built. You now know all of it.

PROVE IT WORKS

Test the whole thing end to end



Sign up as User A

add a few items — they save to Firestore



Log in as User B

empty list — B can't see A's items



Back to User A

items still there — data persisted



Open on your phone

edit on one, it updates live on the other

If all four pass, you've built a genuinely real app: secure, multi-user, and always in sync.

MAKE IT YOURS

Build your own end-to-end app

Same recipe, your idea. Each of these needs login + a database + CRUD + per-user data:



Personal task manager

your to-dos, private to you



Expense tracker

log spending, see totals



Habit tracker

daily ticks + streaks



Notes / journal

private entries, saved



Watchlist

books or films to track



Contact book

your own mini-CRM

DO IT SAFELY

Security best practices



Rules are the real lock

App-side filtering can be bypassed; server rules can't.



Never expose secrets

Keep admin keys out of the app and off public repos.



Least privilege

Users touch only their own data — nothing more.



Strong passwords + verify

Set a password policy; verify emails for real apps.



WEEK 04 COMPLETE

You build real apps now.

Database + CRUD + logins + per-user data + security. From a forgetful demo to a real, multi-user app in one week.

Homework · finish your own end-to-end capstone app. **Next week** — new ground.